DAA LAB 3:

1. PERFORMANCE ANALYSIS OF GREEDY ALGORITHMS

1. Objective

The objective of this experiment is to implement and evaluate three greedy algorithms: Fractional Knapsack, Activity Selection, and Huffman Coding. Their execution time and memory usage are measured for different input sizes, and the experimental results are compared with their theoretical time and space complexities.

2. Experimental Setup

The algorithms were implemented in Python. Execution time was measured using Python's `time.perf_counter()` function, and peak traced memory was measured using the `tracemalloc` module.

Each algorithm was tested with input sizes of 100, 500, 1000, and 5000. Each benchmark was repeated five times, and the average execution time and average peak traced memory were recorded.

3. Execution-Time and Memory Measurements

Insert Table 1 here: Copy the complete table from `results/benchmark_results.csv`. Include all 12 rows and the columns for Algorithm, Input Size, Execution Time (seconds), and Peak Traced Memory (KiB).

The measurements provide an experimental comparison of how execution time and memory usage change as the input size increases. Actual measurements can vary because of system load, Python runtime overhead, and implementation details.

4. Theoretical Complexity Comparison

Algorithm	Theoretical Time Complexity	Auxiliary Space Complexity
Fractional Knapsack	O(n log n)	O(n)
Activity Selection	O(n log n)	O(n)
Huffman Coding	O(L + k log k)	O(k) for tree and supporting structures; O(L) additional for encoded output

Here, n is the number of items or activities, L is the length of the input text, and k is the number of distinct characters.

Fractional Knapsack

The algorithm sorts items by their value-to-weight ratio and then allocates items greedily until the capacity is filled. Sorting dominates the execution time, giving O(n log n) time complexity. The sorted list and selected-item information require O(n) auxiliary space.

Activity Selection

The algorithm sorts activities by finishing time and selects each activity that starts at or after the finish time of the previously selected activity. Sorting dominates the execution time, giving O(n log n) time complexity. The sorted activities and selected list require O(n) auxiliary space.

Huffman Coding

The algorithm counts character frequencies, builds a Huffman tree using a priority queue, and generates binary codes. Frequency counting and encoding take O(L) time, while building the tree takes O(k log k) time. Therefore, the total time complexity is O(L + k log k).

5. Performance Graphs

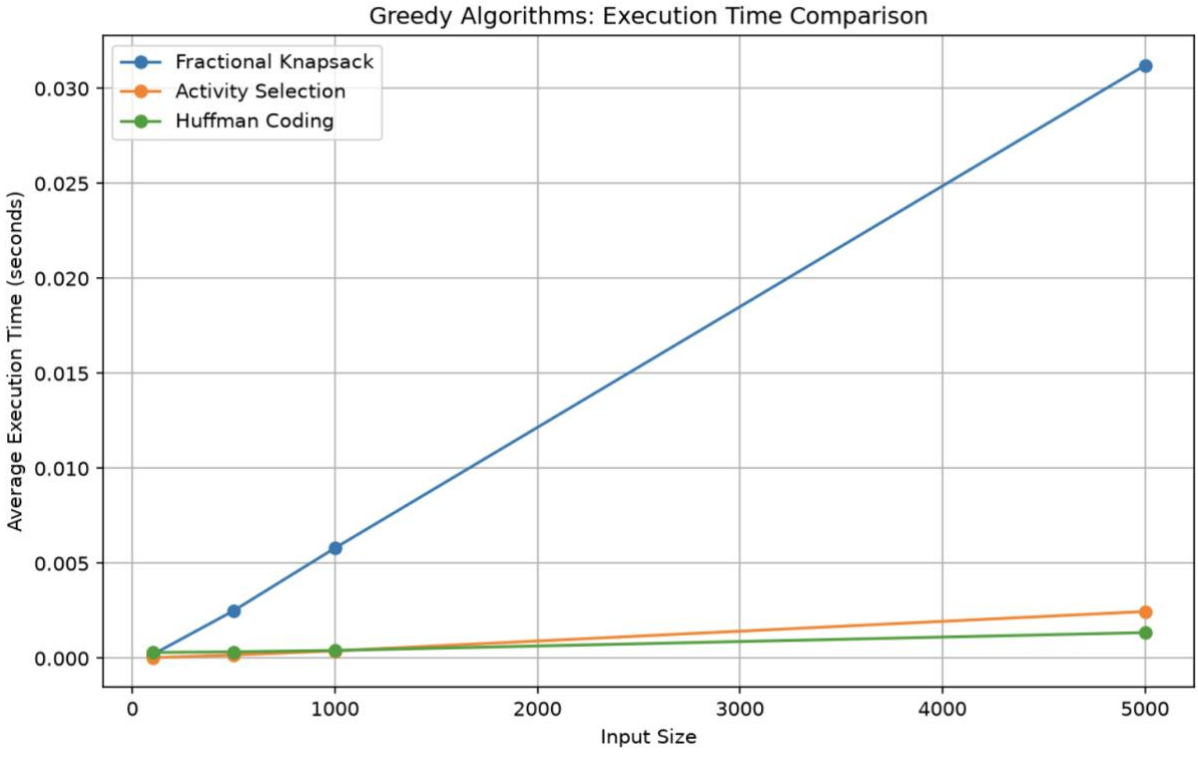


Figure 1. Execution time versus input size for the three greedy algorithms.

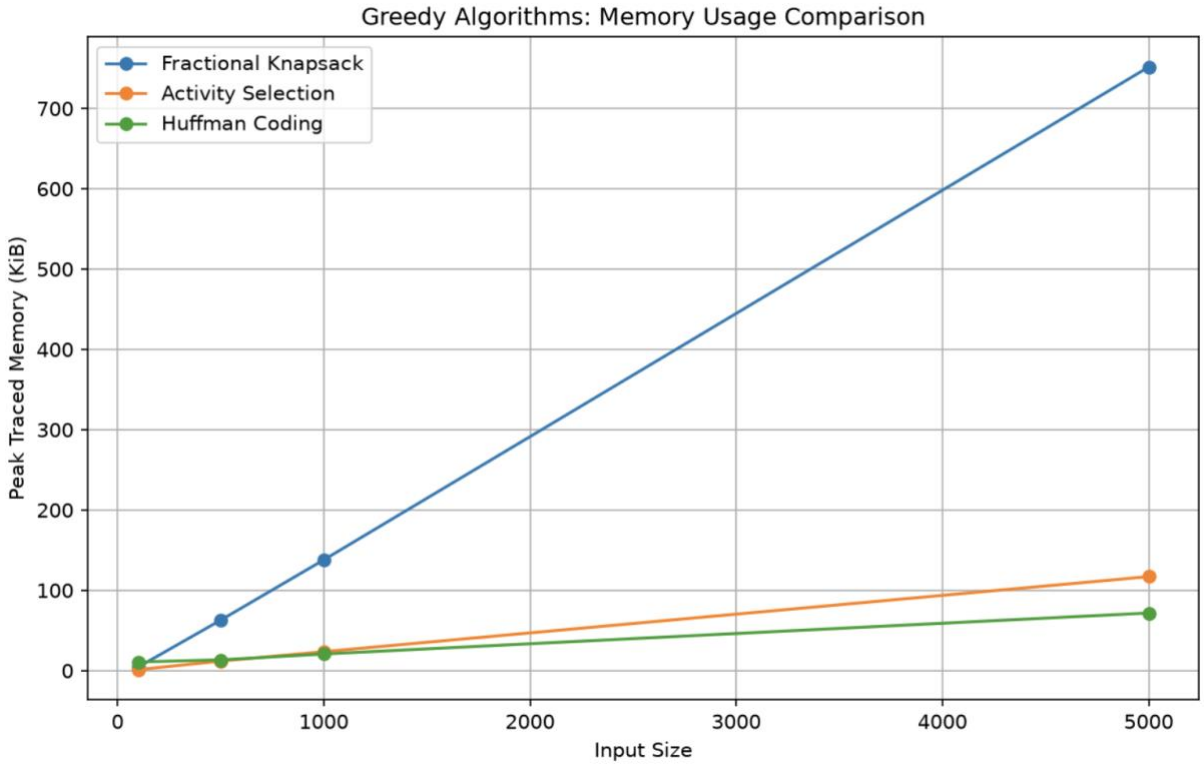


Figure 2. Peak traced memory versus input size for the three greedy algorithms.

6. Analysis and Observations

The execution-time graph helps illustrate the effect of increasing input size on each algorithm. Fractional Knapsack and Activity Selection both perform sorting, so their theoretical time complexity is O(n log n).

Huffman Coding depends on the input text length and the number of distinct characters. The test generator uses 26 lowercase letters, so the number of distinct characters is at most 26. As a result, tree-building costs may remain relatively small compared with processing the text.

The memory graph reports Python-traced allocations rather than the entire Python process's memory usage. Memory usage depends on the input data, temporary collections, tree nodes, and encoded output.

The experimental results should be compared with the theoretical complexities while allowing for variation caused by system conditions and implementation overhead.

7. Conclusion

The experiment demonstrates how execution time and memory usage can be measured for greedy algorithms. Theoretical analysis explains the expected growth rates, while benchmarking and visualization show their observed behaviour for the tested input sizes.